

Enoncés TDs et TP / Angular v ≥ 17 , partie 3b

1. TP facultatif "Devise CRUD"

Le Tp "Devise_CRUD" est un Tp de synthèse facultatif (mais intéressant si on a un peu de temps à y consacrer). L'objectif de ce Tp est de réviser et approfondir (par l'expérience) les principales syntaxes liées aux composants et services Angular (avec appels HTTP en mode GET/POST/PUT et DELETE).

1.1. Phase 1) récupération du point de départ du Tp:

1) copier/coller de `pour_debut_tp` vers votre projet my-app
les parties:

src/app/common/data/devise.ts

src/app/devise (tout ce répertoire avec **devise.component.ts** , **.html** , **.css**)

2) ajouter si besoin la route { path: 'ngr-devise', component: DeviseComponent }
dans **src/app/app-routes.ts** et import qui va avec

3) ajouter si besoin `<a routerLink='/ngr-devise'> devise (crud)</a>  `
dans **src/app/header/header.component.html**

ng serve et `http://localhost:4200`

1.2. Phase 2) Version préliminaire sans accès HTTP

Au sein du fichier **devise.component.ts** récupéré (en tant que point de départ du Tp) :

- **tabDevises** correspond à une liste de devises en mémoire qu'il faut afficher dans le tableau html
- **selectedDevise** correspond à une référence sur une éventuelle sélection de devise (après click sur une des lignes du tableau)
- **deviseTemp** correspond à une copie (créée par clonage) de la devise sélectionnée.
deviseTemp sera affichée dans le formulaire de saisie et sera soit "réinitialisée" , "ajoutée" , "modifiée" ou "supprimée" en fonction du bouton poussoir activé (avec méthodes événementielles `onNew()` , `onAdd` , `onUpdate` , `onDelete()`)
- **sMessage** est un message à construire à destination de l'utilisateur
- **sMode** (vautra "existingOne" suite à une sélection de devise existante ou bien "newOne" suite à un click sur "new") .

L'objectif de cette phase du Tp consiste à coder (par ajout des parties manquantes sur **devise.component.ts** et **.html**) un comportement "CRUD" complet où les valeurs sont pour l'instant qu'actualisées en mémoire au niveau de **this.tabDevises** .

devise sélectionnée (newOne)

code:

name:

change:

Liste des devises (selon serveur)

code	name	change
GBP	Livre	0.844698
EUR	Euro	1
USD	Dollar	1.109257
JPY	Yen	157.875083

message: **devise bien mise à jour**

devise sélectionnée (existingOne)

code:

name:

change:

Ordre d'amélioration conseillé:

- ajouter si besoin (click)="onSelectDevise(d)" sur <tr ... > (dans boucle @for())
- coder si besoin la méthode onSelectDevise()
- ajouter tous les [(ngModel)]="deviseTemp.xyz" manquants dans le côté html
- ajouter si besoin (click)="onUpdate()" sur bouton update
- coder méthode onUpdate() via des instructions de type
this.selectedDevise.xyz = this.deviseTemp.xyz;
- tester tout cela
- coder le reste (new --> remet this.deviseTemp à vide (nouvelle instance de Devise)
(add --> ajoute this.deviseTemp (ou un clone) dans this.tabDevises)
(delete --> supprime this.selectedDevise du tableau this.tabDevises)
- peaufiner message , mise en évidence de l'élément sélectionné et boutons activables ou pas selon le contexte (ex: [style.visibility]="(sMode()=='newOne')?'visible':'hidden'" sur certains boutons du côté .html avec this.sMode.set("newOne") ou bien "existingOne" du côté .ts

1.3. Phase 3) Avec appels HTTP vers api REST

L'objectif de cette phase du Tp consiste à améliorer le code de la phase précédente pour qu'en arrière plan , les actions déclenchées soit synchronisées avec les données d'un serveur/backend REST (URL de base = <https://www.d-defrance.fr/tp/devise-api/v1>) .

URL "public" ou "private" spécifique à ce TP :

```
...  
export class DeviseService {
```

```

private _withoutSecurity = false;

public set withoutSecurity(value:boolean){
  this._withoutSecurity=value;
  this.publicOrPrivateBaseUrl=this._withoutSecurity?this.publicBaseUrl:this.privateBaseUrl;
}

public get withoutSecurity():boolean{
  return this._withoutSecurity;
}

_apiBaseUrl = "tp/devise-api/v1"; //with ng serve --proxy-config proxy.conf.json
publicBaseUrl = `${this._apiBaseUrl}/public`;
privateBaseUrl = `${this._apiBaseUrl}/private`;
publicOrPrivateBaseUrl : string =this.privateBaseUrl; //with security by default
...
}

```

On fera en sorte (au sein de deviseComponent) qu'une case à cocher puisse indirectement contrôler l'état de deviseService.withoutSecurity.

En mode "withoutSecurity" , tous est permis (POST,PUT,DELETE,GET) du coté serveur sans que cela nécessite une authentification/login préalable .

En mode "private" (par défaut sécurisé) , toute action de type POST/PUT/DELETE ne sera acceptée du coté serveur qu'après une authentification préalable (en mode admin , ex : username=admin1) .

NB : on effectuera les premiers tests en mode "withoutSecurity" et on peaufinera que plus tard les aspects sécurisés (authentification, retransmission du token via intercepteur,éventuelle variante en mode OAuth2 , ...).

Changements de code conseillés du coté devise.component.ts:

on s'appuie sur **deviseService = inject(DeviseService) ;**

avec une version plus complète (à améliorer) de src/app/common/service/**DeviseService**

avec méthodes du genre :

```

postDevise$(d :Devise): Observable<Devise>{
  const url = `${this.publicOrPrivateBaseUrl}/devises`;
  return this._http.post<Devise>(url,d /*input envoyé au serveur*/);
}

```

exemple (pour Add) dans devise.component.ts:

```

onAdd(){
  this.deviseService.postDevise$(this.deviseTemp)
    .subscribe(
      { next: (savedDevise)=>{
        this.addClientSide(savedDevise); } ,
      error: (err)=>{ this.sMessage.set(messageFromError(err,"echec post")); }
    );
}

```

```
addClientSide(savedDevise:Devise){  
  this.tabDevises.push(savedDevise);  
  this.onNew(); this.sMessage.set("devise ajoutée")  
}
```

avec *messageFromError()* et *messageFromEx()* fonctions utilitaires de *src/app/common/util/**util.ts*** .

..à peaufiner .. et à compléter sur les autres modes GET/PUT/DELETE/...
..on pourra éventuellement s'appuyer sur des syntaxes de type *async/await* ...